Automated Web Scanner: Project Report

Auditing HTTP Security Headers, CORS Flaws & Information Disclosure

> NOTE
>
> This project report documents the design, architecture, implementation, and empirical verification of **WebScanner**, an automated Python tool designed to evaluate web application defensive posture against OWASP standards.

1. Executive Summary

Modern web applications are distributed across heterogeneous infrastructures, multi-tier architectures, and third-party microservices. While modern frontend frameworks and API backends provide rapid delivery, defensive hardening at the HTTP transport and browser enforcement layers is frequently neglected or misconfigured. Missing security headers and misconfigured Cross-Origin Resource Sharing (CORS) policies are among the most prevalent vulnerabilities reported across automated enterprise assessments and bug bounty programs.

The **WebScanner** project is an automated, modular, and extensible command-line security auditing solution written in Python. It performs in-depth security posture assessments against any target URL or batch of targets. The scanner audits critical HTTP security headers according to OWASP guidelines, detects technology stack information leakage, actively probes for dangerous CORS flaws (such as arbitrary origin reflection, null origin trust, and credential theft risks), and computes an executive security grade (`A+` through `F`) with actionable remediation guidance.

2. Problem Statement & Theoretical Background

2.1 The Browser Security Model

Browsers execute client-side code in sandboxed contexts governed by the **Same-Origin Policy (SOP)**. However, SOP alone does not protect against clickjacking, MIME confusion attacks, cross-site script injection (XSS), or man-in-the-middle SSL stripping. Web application servers must explicitly declare defensive constraints via HTTP response headers.

2.2 Security Headers Audited

Security Header	Security Purpose	Recommended Baseline
Strict-Transport-Security (HSTS)	Enforces HTTPS connections; eliminates SSL stripping/MITM attacks.	`max-age >= 31536000; includeSubDomains; preload`
Content-Security-Policy (CSP)	Restricts script, style, and object sources to mitigate Cross-Site Scripting (XSS).	`default-src 'self'; frame-ancestors 'none';`
X-Frame-Options	Prevents clickjacking by disabling unauthorized framing in iframes.	`DENY` or `SAMEORIGIN`
X-Content-Type-Options	Prevents MIME-sniffing attacks where browsers execute non-script files as scripts.	`nosniff`
Referrer-Policy	Controls referrer leakage of sensitive query parameters and URL tokens.	`strict-origin-when-cross-origin`
Permissions-Policy	Restricts browser feature delegation (camera, microphone, geolocation).	`camera=(), microphone=(), geolocation=()`
Cross-Origin-Opener-Policy (COOP)	Isolates browsing contexts from cross-origin windows to mitigate Spectre/leakage.	`same-origin`

2.3 Cross-Origin Resource Sharing (CORS) Flaws

CORS is a relaxation of the Same-Origin Policy designed to allow legitimate cross-domain requests. However, developers frequently introduce critical security oversights when attempting to resolve CORS errors:

1. **Arbitrary Origin Reflection**: The server reads the incoming request's `Origin` header and dynamically copies it into `Access-Control-Allow-Origin`. When combined with `Access-Control-Allow-Credentials: true`, any malicious site can forge requests and steal authenticated session data.
2. **Wildcard Origin with Credentials**: Setting `Access-Control-Allow-Origin: *` alongside `Access-Control-Allow-Credentials: true`. Modern browsers reject this combination, but it reveals flawed security logic and may expose sensitive APIs to uncredentialed reads.
3. **Null Origin Trust (`Origin: null`)**: Sandboxed iframes (`<iframe sandbox='allow-scripts'>`) and local file documents send `Origin: null`. Servers that trust `null` can be exploited by an attacker embedding a sandboxed frame on an external site.
4. **Sensitive Exposed Headers**: Revealing `Set-Cookie` or `Authorization` in `Access-Control-Expose-Headers`.

3. System Architecture & Component Design

Component Breakdown

Module	Role	Key Responsibilities
	Rule Configuration	Defines header specs, validation regexes, severity levels, test origins, and scoring penalties.
	Network Layer	Manages `requests.Session`, timeouts, custom User-Agents, SSL verification toggles, and redirect chains.
	Audit & Analysis Engine	Performs static header evaluation, technology disclosure detection, and active CORS probing.
	Presentation & Serialization	Formats colorized terminal outputs and exports machine-readable JSON and text reports.
	Diagnostics	Centralized logging to stderr and persistent log files (`webscanner.log`) without polluting CLI outputs.
	CLI & Orchestration	Handles CLI arguments (`argparse`), interactive prompts, input validation, and batch execution loops.
	Quality Assurance	Automated mock test suite validating detection accuracy across various simulated responses.

4. Scoring Algorithm & Rating Scale

WebScanner starts every evaluated host with a perfect baseline of **100 points**. Penalties are deducted based on findings and their associated severity:

```
Score = 100 - SUM(Severity Penalties)
```

Severity Level	Penalty	Example Qualifying Condition
CRITICAL	**-30 pts**	Arbitrary origin reflection with credentials; Wildcard origin with credentials
HIGH	**-15 pts**	Missing HSTS on HTTPS; Missing CSP; Unencrypted plaintext HTTP transport
MEDIUM	**-8 pts**	Missing X-Frame-Options; Missing X-Content-Type-Options; Weak CSP directives
LOW	**-4 pts**	Missing Referrer-Policy; Missing Permissions-Policy; HSTS max-age < 6 months
INFO	**-1 pt**	Server banner present; CMS generator disclosure

Executive Grade Mapping

- **A+ (95 - 100)**: Exemplary defensive posture with robust headers, strict CSP, and locked-down CORS.
- **A (85 - 94)**: Strong security posture with minor non-critical omissions (e.g., Permissions-Policy).
- **B (70 - 84)**: Moderate security posture; core headers present but contains weak directives or info disclosure.
- **C (50 - 69)**: Substandard posture; lacks multiple essential defenses (e.g., clickjacking or CSP).
- **D (30 - 49)**: Poor defensive posture; multiple high-severity omissions or plain HTTP transport.
- **F (< 30)**: Critical vulnerabilities detected, complete lack of defenses, or connection failure.

5. Automated Verification & Testing

A dedicated test suite () was implemented using Python's `unittest` module and `unittest.mock`.

```
Ran 6 tests in 0.004s

OK
```

Test Case Coverage

1. `test_ideal_secure_headers`: Confirms that an endpoint with all recommended headers receives 100 points and Grade `A+`.
2. `test_missing_security_headers_http`: Confirms that an unencrypted HTTP target missing all headers is heavily penalized (Grade `D`).

3. `test_cors_wildcard_with_credentials`: Confirms `*` + `credentials=true` triggers a `CRITICAL` finding.
4. `test_cors_dynamic_origin_reflection_with_credentials`: Confirms dynamic reflection triggers a `CRITICAL` finding.
5. `test_cors_null_origin_reflection`: Confirms `null` origin acceptance triggers a `CRITICAL` finding.
6. `test_information_disclosure_detection`: Confirms detection and extraction of `Server`, `X-Powered-By`, and `X-AspNet-Version` headers.

6. Empirical Results & Real-World Validation

WebScanner was executed against production websites to validate live behavior, redirect handling, and header analysis accuracy:

Target Endpoint	Score	Grade	Key Findings
`https://example.com`	42 / 100	**Grade D**	Missing HSTS, CSP, X-Frame-Options, X-Content-Type-Options; Server: `cloudflare`
`https://google.com`	50 / 100	**Grade C**	`X-Frame-Options: SAMEORIGIN` present; Missing HSTS (redirect host), CSP; Server: `gws`
`https://github.com`	84 / 100	**Grade B**	HSTS active (1 yr + subdomains + preload), XFO `DENY`, nosniff; CSP contains `'unsafe-inline'`

7. Hardening Guidelines

Nginx Server Configuration (`nginx.conf`)

```
nginx
# Enforce HSTS (1 year with subdomains and preload)
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; preload" always;

# Prevent Clickjacking
add_header X-Frame-Options "DENY" always;

# Prevent MIME-type Sniffing
add_header X-Content-Type-Options "nosniff" always;

# Restrict Referrer Information
add_header Referrer-Policy "strict-origin-when-cross-origin" always;

# Content Security Policy
add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self'
'unsafe-inline'; object-src 'none'; base-uri 'self'; frame-ancestors 'none';" always;

# Restrict Browser Feature Delegation
add_header Permissions-Policy "camera=(), microphone=(), geolocation=(), payment=()"
always;

# Suppress Server Token
server_tokens off;
```

Secure CORS Whitelist Implementation (Express.js)

```
javascript
const express = require('express');
const cors = require('cors');
const app = express();

const whitelist = ['https://trusted-app.example.com', 'https://admin.example.com'];

const corsOptions = {
  origin: function (origin, callback) {
    if (whitelist.indexOf(origin) !== -1) {
      callback(null, true);
    } else {
      callback(new Error('CORS request rejected: Origin not allowed.'));
    }
  },
  credentials: true,
  methods: ['GET', 'POST'],
  allowedHeaders: ['Content-Type', 'Authorization'],
  maxAge: 86400
};

app.use(cors(corsOptions));
```

8. Conclusion & Future Roadmap

WebScanner provides a high-performance, developer-friendly solution for proactive web security assessment. By automating the discovery of HTTP header gaps, CORS reflection flaws, and infrastructure leaks, it enables engineering teams to resolve high-impact vulnerabilities prior to production deployment.

Future Roadmap

- **CI/CD Automation**: GitHub Action and GitLab CI templates to fail builds if security scores drop below a configurable threshold.
- **Headless Browser Execution**: Incorporate Playwright/Selenium to detect client-side DOM-based CORS reads and CSP violation reports in real time.
- **Subdomain Discovery Integration**: Support automatic enumeration of subdomains to audit an organization's entire digital footprint.

- **Executive PDF Dashboards**: Extend reporter to generate styled PDF reports with vulnerability charts.